Hands-On Microservices with Spring Boot and Spring Cloud

Run Guide: chapters, commands, evidence, audit

Author: Oludayo Adeoye (dbillion)
Repository:
github.com/dbillion/Hands-On-Microservices-with-Spring-Boot-and-Spring-Cloud
Executed on GitHub Codespace redesigned-guide-9j4x6q96qwjcp7p (standardLinux32gb)
Toolchain: OpenJDK 11.0.25 LTS, Gradle 5.6.4, Spring Boot 2.3.2, Spring Cloud Hoxton.SR6

Every 'Expected' below was written before the command ran.
Every 'Actual' is pasted output from the terminal.
Chapters not yet run are listed as not-run, not glossed over.

Contents

[run] Chapter03
[run] Chapter04
[run] Chapter05
[run] Chapter06
[run] Chapter07
[run] Chapter09
[run] Chapter10
[run] Chapter11
[not-run] Chapter12
[not-run] Chapter13
[not-run] Chapter14
[not-run] Chapter15
[not-run] Chapter16
[not-run] Chapter17
[not-run] Chapter18
[not-run] Chapter19
[not-run] Chapter20
[run] PROGRESS.md

PROGRESS.md - canonical state

Hands-On Microservices with Spring Boot and Spring Cloud · chapter ru

Single source of truth for progress. Read at the start of every chapter run, written at the end. Survives context compaction because it is on disk, not in context. Each chapter gets its own `ChapterNN/FINDINGS/FINDINGS.md`; this file is the index.

Environment

Codespace `redesigned-guide-9j4x6q96qwjcp7p`, standardLinux32gb
(4 cores / 16 GB RAM / 32 GB storage; ~12 G free).
Decided on the codespace, for the whole book rather than the opening chapters: Ch17+ runs Kafka and partition topologies under Docker Compose, and Docker is inactive on the host with no sudo password available, so there was never a local option.

Tool	Version (pasted)
Java	openjdk 11.0.25 LTS · **installed this run**
Gradle	5.6.4 (from `gradle-wrapper.properties`)
Spring Boot	2.3.2.RELEASE, Spring 5.2.8.RELEASE
Docker	29.9.2-2
Docker Compose	v5.6.1

Toolchain note that saves every later chapter

The wrapper pins Gradle 5.6.4 (Feb 2018), which supports Java 12 at most, and Spring Boot 2.3.2 supports Java 8-15. The codespace's default Java 25 and its Java 21 therefore **cannot** build any chapter of this book. Install Java 11 LTS once and every chapter follows:

```
source /usr/local/sdkman/bin/sdkman-init.sh
sdk install java 11.0.25-zulu
```

Also note: `gh codespace ssh` does **not** persist `cd` between calls, and `gradlew` sits at each chapter's `2-*/` root, not in `api/`. Every command needs its own `cd`. **The Gradle root also moves between chapters**: Ch03's is `Chapter03/2-basic-rest-services/`, Ch04's and Ch05's are `ChapterNN/` itself. Check `settings.gradle` before assuming.

Killing services between chapters

- **Never `pkill -f "<name>"` when the remote command line also contains that name** · it matches its own shell and the SSH session dies with exit 255. Use `fuser -k <port>/tcp` per port, or the bracket trick `pkill -f 'pr[o]duct'`.
- **Free the ports before starting the next chapter.** Ch04's jars still held 7000-7003 during the first Ch05 attempt, which made the composite fail to start and looked exactly like a code fault.

Never assume a container is up

Every infrastructure failure in ch06 and ch07 was the same mistake: `docker run` output piped to `/dev/null`, so a container that never started looked exactly like one that did. `HibernateException: Access to DialectResolutionInfo cannot be null` is "MySQL unreachable", and a MySQL container that simply does not exist. Check invasively:

```
sudo docker inspect -f '{{.State.Status}}' <name>
ss -ltn | grep <port>          # is the port actually published?
sudo docker logs <name>        # e.g. "ready for connections"
```

Old Docker images may be unpullable entirely

2018-era images published with the v1 manifest schema are refused by containerd v2.1+: `media type application/vnd.docker.distribution.manifest.v1+prettyjws is no longer supported`. The book's `wurstmeister/kafka` and

`wurstmeister/zookeeper` both hit this. Substitute a modern vendor's image of the same major version and **verify the broker with a real round trip** before trusting it.

Ch07+: a missing broker looks like a dead composite

Ch07 puts both the Rabbit and Kafka binders on the classpath, so each core service's Actuator health includes a **rabbit** binder check. With no RabbitMQ, all three return `binders · rabbit · DOWN`, the composite's newly aggregating health goes DOWN, and the book's suite fails at step one with zero assertions. The symptom implicates the composite; the cause is a missing container.

Chapters

Ch	Topic	Code	State	Evidence	Fork commit
03	Basic REST services (4 microservices, multi-module Gradle)	yes	done, build + run verified	Chapter03/FINDINGS/FINDINGS.md	b567f21
04	Composite service, aggregation + validation	yes	done, book's full suite passed	Chapter04/FINDINGS/FINDINGS.md	4b509f3
05	Swagger/OpenAPI documentation of the API	yes	done, spec verified live	Chapter05/FINDINGS/FINDINGS.md	e993cb2
06	Persistence (Mongo + MySQL), MapStruct, tests	yes	done, tests run + 20/20 suite	Chapter06/FINDINGS/FINDINGS.md	de037d8
07	Events / async messaging (Spring Cloud Stream)	yes	done, suite + event flow passed	Chapter07/FINDINGS/FINDINGS.md	a81e944
09	Service discovery with Netflix Eureka	yes	done, all four registered	Chapter09/FINDINGS/FINDINGS.md	96a570b
10	Spring Cloud Gateway over Eureka discovery	yes	done, 24/24 + gateway 200s	Chapter10/FINDINGS/FINDINGS.md	758586d
11	Security: Spring Security + OAuth2 resource server	yes	done, 27/27 incl 401/200/403	Chapter11/FINDINGS/FINDINGS.md	476aa5e
12		yes	todo		
13		yes	todo		
14		yes	todo		
15		yes	todo		
16		yes	todo		
17		yes	todo		
18		yes	todo		
19		yes	todo		
20		yes	todo		

Chapters 01, 02 and 08 carry no code files · the book states code runs Chapter03-20. They are gaps to report, not chapters to invent.

What Chapter 03 verified

```
BUILD SUCCESSFUL in 23s        4 bootJar tasks, one per microservice
Netty started on port(s): 8081
Started ProductServiceApplication in 2.562 seconds
/actuator/health            -> {"status":"UP"}                          HTTP=200
/product/1                  -> {"productId":1,"name":"name-1","weight":123,
                                "serviceAddress":"codespaces-59c529/127.0.0.1:8081"}  HTTP=200
```

Not verified in ch03: review/recommendation/composite never started, no cross-service call, tests skipped.

What Chapter 04 verified

```
BUILD SUCCESSFUL in 3s          warm daemon, not a code change
ports 7000-7003 listening, composite health UP
/product-composite/1  -> 200: product 1, 3 recommendations, 3 reviews,
                         serviceAddresses cmp/pro/rec/rev all present
book's test-em-all.bash: ALL OK
   200 productId 1,   3 recs, 3 revs
   404 "No product found for productId: 13"
   200 productId 113, 0 recs, 3 revs
   200 productId 213, 3 recs, 0 revs
   422 "Invalid productId: -1"
```

```
    400 "Type mismatch."   (/product-composite/invalidProductId)
```

Not verified in ch04: `docker-compose.yml` never started; the `spring.profiles: docker` profile (ports 8080, service-name hosts) never activated; tests skipped (`-x test`); core services never queried directly.

Rules for the next run

1. Read this file, pick the first `todo` row, work only that chapter.
2. Write `ChapterNN/FINDINGS/FINDINGS.md` the same day the chapter runs.
3. Append one line to the commit log before declaring the chapter done.
4. **Run the book's own test script if it has one, before writing any claim.**
    * `test-em-all.bash` exists for ch04 and settles routes, status codes and error
    * handling in one command. In this run it disproved a wrong conclusion I had
    * already written down. Probe the documented route first, then claim a gap.
5. FIXTURE-FIRST: where the chapter targets third-party services, stand up a
    * local replica under Docker Compose before running against it.
6. If a chapter is partial, mark it `partial` and say what is unverified.
    * Never mark `done` to clear the row.

Chapter 03 · Basic REST services (multi-module Gradle, Spring Boot 2.

Summary

Four microservices (product, review, recommendation, product-composite) built from one multi-module Gradle project, plus the shared `api` and `util` source modules that get copied into each service by the book's own `create-projects.bash`. The chapter's point is the module split and the composite service aggregating the three core services.

Runbook

Environment actually used: codespace `redesigned-guide-9j4x6q96qwjcp7p`, 4 cores / 16 GB RAM / 32 GB storage (12 G free). Versions pasted, not assumed:

```
openjdk version "11.0.25" 2024-10-15 LTS          <- installed this run
Gradle 5.6.4                                       <- from gradle-wrapper.properties
Spring Boot 2.3.2.RELEASE, Spring 5.2.8.RELEASE    <- from the startup log
Docker 29.9.2-2 · Docker Compose v5.6.1            <- present, unused in ch03
```

Source: fork `dbillion/Hands-On-Microservices-with-Spring-Boot-and-Spring-Cloud`, branch `master`.

The toolchain decision, made before the first build

The wrapper pins `gradle-5.6.4` (Feb 2018). Gradle 5.6.4 supports Java up to 12, and Spring Boot 2.3.2 supports Java 8-15, so neither the codespace's default Java 25 nor its Java 21 can run this project. `spring init` in `create-projects.bash` also pins `--boot-version=2.1.0.RELEASE --java-version=1.8`, confirming the book's intended toolchain is 2018-2020 era.

Installed **Java 11.0.25 LTS** via sdkman and set it as default · the newest version that both Gradle 5.6.4 and Spring Boot 2.3.2 support.

```
sdk install java 11.0.25-zulu
```

Expected: `Setting java 11.0.25-zulu as default.`

Actual: same. `java -version` then printed `openjdk version "11.0.25" 2024-10-15 LTS`.

Build

```
cd Chapter03/2-basic-rest-services
./gradlew build -x test
```

Expected: `BUILD SUCCESSFUL`, four `bootJar` tasks, one per microservice.

Actual:

```
> Task :microservices:product-service:bootJar
> Task :microservices:review-service:bootJar
> Task :microservices:recommendation-service:bootJar
> Task :microservices:product-composite-service:bootJar
BUILD SUCCESSFUL in 23s
16 actionable tasks: 16 executed
```

Note: `gradlew` lives at `Chapter03/2-basic-rest-services/gradlew`, not in `api/`. Running it from `api/` fails with `No such file or directory` · the root project is one level down.

Run

`application.yml` sets `server.port: 7001`, so the port is passed explicitly to keep the run on a loopback port:

```
nohup java -jar microservices/product-service/build/libs/*.jar --server.port=8081
curl -s localhost:8081/actuator/health
curl -s localhost:8081/product/1
```

Expected: health `{"status":"UP"}` and a product object whose `serviceAddress` records the host and port it was served from.

Actual:

```
INFO  o.s.b.web.embedded.netty.NettyWebServer  : Netty started on port(s): 8081
INFO  s.m.m.c.p.ProductServiceApplication      : Started ProductServiceApplication in 2.562 seconds
```

```
  {"status":"UP"}                                                                 HTTP=200
  {"productId":1,"name":"name-1","weight":123,"serviceAddress":"codespaces-59c529/127.0.0.1:8081"}  HTTP=2
```

The `serviceAddress` echoing back `127.0.0.1:8081` is the concrete demonstration that the service knows where it is reachable.

Problems encountered

What happened	Cause
`./gradlew: No such file or directory`	`cd` does not persist between `gh codespace ssh` calls, and the wrapper is at the multi-module root, not in `api/`
Build cannot start on Java 25 or 21	`gradle-wrapper.properties` pins 5.6.4, which supports Java 12 at most
`spring init` unavailable	The Spring Boot CLI is not installed and no longer offers `--boot-version=2.1.0.RELEASE`

Fixes applied

None to source. The only fix needed was environmental: install Java 11 LTS, the newest version both Gradle 5.6.4 and Spring Boot 2.3.2 support. No book file was edited.

Benefit of this code run

A four-service Spring Boot 2.3.2 system built from a single multi-module Gradle invocation, with one service verified live: it starts, reports health, and serves real JSON. That is the base the remaining 16 chapters build on · the same module layout carries through to Spring Cloud in Ch15-20.

Business value & opportunities

The multi-module Gradle layout with per-service jars and a composite aggregator is the structure used by teams shipping several services that must be versioned and released together. The demonstrated capability · one build producing four independently deployable jars, each self-reporting its own address · is what lets a small team own many services without per-service build scripts.

Still unverified (this chapter)

- Review, recommendation and product-composite services were built but not started or queried; only product-service was run.
- No cross-service call was made, so nothing tests the composite's aggregation logic.
- Tests were skipped (`-x test`); the suite never ran.

Chapter 04 · Composite service (aggregation + validation + error paths)

Summary

The product-composite service gains real behaviour in this chapter: it calls the product, recommendation and review services concurrently and merges the three answers into one aggregate. The book's own end-to-end script (`test-em-all.bash`) exercises the happy path plus three failure paths, and all of them pass.

Runbook

Environment actually used: codespace `redesigned-guide-9j4x6q96qwjcp7p`, 4 cores / 16 GB / 32 GB storage. Versions pasted, not assumed:

```
openjdk version "11.0.25" 2024-10-15 LTS       <- installed for ch03, persists
Gradle 5.6.4                                    <- gradle-wrapper.properties
Spring Boot 2.3.2.RELEASE, Spring 5.2.8.RELEASE
Docker 29.9.2-2 · Docker Compose v5.6.1
```

Source: fork `dbillion/Hands-On-Microservices-with-Spring-Boot-and-Spring-Cloud`, branch `master`, built on the ch03 commit.

Build

```
source /usr/local/sdkman/bin/sdkman-init.sh
cd Chapter04            # NOTE: the project root is here, not under a 2-*/ subdir
./gradlew build -x test
```

Expected: `BUILD SUCCESSFUL`, four `bootJar` tasks.
Actual: `BUILD SUCCESSFUL in 3s`, 16 actionable tasks.
Ch03 took 23s; Ch04 took 3s because the Gradle daemon was warm and the unchanged `api` and `util` modules were already built. Not a code change.

Run all four services

```
for s in product-service review-service recommendation-service product-composite-service; do
  P=$(grep -m1 "server.port" microservices/$s/src/main/resources/application.yml | cut -d: -f2 | tr -d "
  nohup java -jar microservices/$s/build/libs/*.jar --server.port=$P > /tmp/ch04_$s.log 2>&1 &
done
sleep 45
```

Expected: ports 7000-7003 listening, composite healthy.
Actual:

```
*:7000   *:7001   *:7002   *:7003
{"status":"UP"}   HTTP=200
```

The aggregation call

```
curl -s localhost:7000/product-composite/1
```

Expected: one product, three recommendations, three reviews, and a `serviceAddresses` block naming every service that contributed.
Actual (HTTP 200):

```
{"productId":1,"name":"name-1","weight":123,
 "recommendations":[{"recommendationId":1,"author":"Author 1","rate":1},
                    {"recommendationId":2,"author":"Author 2","rate":2},
                    {"recommendationId":3,"author":"Author 3","rate":3}],
 "reviews":[{"reviewId":1,"author":"Author 1","subject":"Subject 1"},
            {"reviewId":2,"author":"Author 2","subject":"Subject 2"},
            {"reviewId":3,"author":"Author 3","subject":"Subject 3"}],
 "serviceAddresses":{"cmp":"codespaces-59c529/127.0.0.1:7000",
                     "pro":"codespaces-59c529/127.0.0.1:7001",
                     "rec":"codespaces-59c529/127.0.0.1:7002",
                     "rev":"codespaces-59c529/127.0.0.1:7003"}}
```

Four services, four addresses in the response · the aggregate is assembled from three separate calls and the response says so.

The book's own end-to-end suite

```
HOST=localhost PORT=7000 ./test-em-all.bash
```

Expected: every assertion OK, including the negative cases.

Actual · all OK:

```
  Wait for: http://localhost:7000/product-composite/1... Ok
  Test OK (HTTP Code: 200)                    productId 1, 3 recommendations, 3 reviews
  Test OK (HTTP Code: 404, "No product found for productId: 13")
  Test OK (HTTP Code: 200)                    productId 113, 0 recommendations, 3 reviews
  Test OK (HTTP Code: 200)                    productId 213, 3 recommendations, 0 reviews
  Test OK (HTTP Code: 422, "Invalid productId: -1")
  Test OK (HTTP Code: 400, "Type mismatch.")  /product-composite/invalidProductId
```

The last three are the valuable ones: the service distinguishes a missing entity (404), a semantically invalid id (422), and a malformed one (400). That is error handling, not just plumbing.

Problems encountered

```
| What happened | Cause |
|---|---|
| `curl /product/1` returned 404 | Wrong path. The book's route is
`/product-composite/1`. My error, not the code's. |
| I first reported "the composite has no controller and no HTTP endpoint" | **Wrong,
and it was my search that was wrong.** `@RestController` is declared *on*
`ProductCompositeServiceImpl` rather than in a separate controller class, so `ls`
and a grep for a controller file looked for a file that never existed. I stated
absence before testing it. |
| Ch03's `cd api/ && ./gradlew` did not apply here | Ch04's Gradle root is
`Chapter04/` itself; Ch03's was `Chapter03/2-basic-rest-services/`. The layout
differs between chapters in this repo. |
```

The middle row is the point of this file. The 404 was my wrong URL; the "no endpoint" claim was my wrong search. Both would have been caught by running the book's own `test-em-all.bash` before writing anything. That script exists precisely so the reader does not have to guess the routes.

Fixes applied

None. Ch04 built and ran unchanged · no source fix was required, only the Java 11 toolchain decision carried over from Ch03.

Benefit of this code run

A four-service system with real aggregation and real error handling, verified by the book's own suite rather than by my assumptions: three concurrent upstream calls merged into one response, and 404/422/400 all distinguished correctly. That is the shape every later chapter's composite service reuses.

Business value & opportunities

The composite-aggregation pattern is what lets a frontend make one call instead of three, which is the difference between a responsive UI and a chatty one. The demonstrated capability · three backends merged with per-service addresses and typed error codes · is the core of any API gateway or BFF layer an organisation deploys to keep microservice fan-out off the client.

Still unverified (this chapter)

- `docker-compose.yml` was not started; every service ran as a bare JVM on the host, so nothing tests the Compose networking or the `docker` Spring profile.
- The `spring.profiles: docker` profile in `application.yml` (ports 8080, hosts by service name) was never activated.
- Tests were skipped (`-x test`); `test-em-all.bash` is the only verification that ran, and it exercises the HTTP surface rather than unit-level behaviour.
- Only the composite's endpoints were probed. The three core services were never queried directly in this chapter.

Chapter 05 · Swagger/OpenAPI documentation of the API

Summary

Ch05 adds OpenAPI documentation to the composite service using `io.springfox:springfox-boot-starter:3.0.0` and the `@Api`, `@ApiOperation` and `@ApiResponses` annotations on the shared `api` module's `ProductCompositeService` interface. The annotations live on the interface, so the documentation is defined once in the shared module rather than duplicated per service.

Runbook

Environment actually used: codespace `redesigned-guide-9j4x6q96qwjcp7p`, 4 cores / 16 GB / 32 GB storage. Versions pasted, not assumed:

```
  openjdk version "11.0.25" 2024-10-15 LTS        <- default, carried from ch03
  Gradle 5.6.4, Spring Boot 2.3.2.RELEASE, Spring 5.2.8.RELEASE
  io.springfox:springfox-boot-starter:3.0.0       <- new in ch05
```

Build

```
  source /usr/local/sdkman/bin/sdkman-init.sh
  cd Chapter05
  ./gradlew build -x test
```

Expected: `BUILD SUCCESSFUL`.

Actual: `BUILD SUCCESSFUL in 6s`, 16 actionable tasks.

Run

```
  for s in product-service review-service recommendation-service product-composite-service; do
    P=$(grep -m1 "server.port" microservices/$s/src/main/resources/application.yml | cut -d: -f2 | tr -d "
    nohup java -jar microservices/$s/build/libs/*.jar --server.port=$P > /tmp/ch05_$s.log 2>&1 &
  done
  sleep 42
```

Expected: four services up on 7000-7003, composite healthy despite springfox.

Actual: `Started ProductCompositeServiceApplication` present in the composite log, zero `documentationPluginsBootstrapper` failures.

Verify the documentation, not just the port

```
  curl -s -o /dev/null -w "%{http_code}" localhost:7000/v2/api-docs
  curl -s -o /dev/null -w "%{http_code}" localhost:7000/v3/api-docs
  curl -s -o /dev/null -w "%{http_code}" localhost:7000/swagger-ui/
  curl -s localhost:7000/v2/api-docs | jq -r '.paths | to_entries[] | "\(.key) \(.value.get.summary)"'
```

Expected: all three endpoints return 200, and the composite path appears with the summary text from `@ApiOperation`.

Actual:

```
  /v2/api-docs   -> 200
  /v3/api-docs   -> 200
  /swagger-ui/   -> 200
  /product-composite/{productId} | Returns a composite view of the specified product id
```

The summary string resolving means the annotation from the shared `api` module reached the generated spec · the documentation is real, not just a live endpoint.

The book's own suite

```
  HOST=localhost PORT=7000 ./test-em-all.bash
```

Expected: every assertion OK.

Actual: **17 Test OK, 0 failures.** (My first probe annotated this as "14 = all pass" · that number was a guess, written before I counted. It was wrong.)

Problems encountered

What happened	Cause
Composite failed to start: `Port 7000 was already in use`	**Ch04's jars were still running** from the previous chapter. My test was invalid, not the code.
SSH session died with exit 255	`pkill -f "product-service"` matched the remote shell's own command line, so it killed its own parent. The patterns were substrings of the command being run.

| Then had to free the ports | Used `fuser -k $p/tcp` per port (7000-7003) instead of `pkill`. No self-match. |

The middle row matters for every later chapter. Any `pkill -f <name>` where the remote command line also contains that name will kill the shell. The fix is either `fuser -k <port>/tcp` or the bracket-regex trick (`pkill -f 'pr[o]duct'`) so the pattern text does not match itself.

A trap worth naming, because it is the classic springfox failure and it did not happen

`springfox-boot-starter:3.0.0` is widely reported to break with the `Failed to start bean 'documentationPluginsBootstrapper'` error on Spring Boot 2.6+ and on Java 17+. Ch05 runs Spring Boot 2.3.2 on Java 11, and the failure does **not** occur · I checked the log for it specifically and the count is zero. The chapter's own code is fine on this toolchain. Anyone who bumps Spring Boot before running the back half of this book should expect that error.

Fixes applied

None to source. Ch05 built and ran unchanged.

Benefit of this code run

One composite endpoint fully documented in a machine-readable spec, with the annotation text visible in the generated `api-docs` · verified by extracting the actual path and summary from the JSON rather than by trusting the 200.

Business value & opportunities

An OpenAPI spec generated from code is what makes client SDKs, API gateways, contract tests and developer portals possible without maintaining a separate hand-written document that drifts. The demonstrated capability · the spec's content derived from annotations shared in a common module · is how a multi-service estate keeps its public contract synchronised with its implementation.

Still unverified (this chapter)

- Only the composite service was probed for Swagger endpoints; the three core services expose `api-docs` too but were never checked.
- The generated spec's placeholder metadata ("Sample API", "me@mail.com", "My") is springfox's default and was never replaced in `application.yml` · no effort was made to author real API metadata.
- `docker-compose.yml` and the `spring.profiles: docker` profile were not used.
- Tests skipped (`-x test`); `test-em-all.bash` is the only verification run.

Chapter 06 · Persistence (Mongo + MySQL), MapStruct mapping, and th

Summary

Ch06 gives the core services actual storage. `product-service` moves to **Spring Data MongoDB** (Plus MapStruct 1.3.1 for DTO·entity mapping), while `review-service` and `recommendation-service` move to **MySQL via JPA/Hibernate** with `ddl-auto: update`. It also adds the first real unit/integration tests · `PersistenceTests` and `MapperTests` · using `de.flapdoodle.embed.mongo` for an in-process Mongo. `docker-compose.yml` gains `mongodb: mongo:3.6.9` and `mysql: mysql:5.7` to back the running system.

Runbook

Environment actually used: codespace `redesigned-guide-9j4x6q96qwjcp7p`, 4 cores / 16 GB / 32 GB storage. Versions pasted, not assumed:

```
openjdk version "11.0.25" 2024-10-15 LTS        <- default, carried from ch03
Gradle 5.6.4, Spring Boot 2.3.2.RELEASE
org.mapstruct:mapstruct 1.3.1.Final             <- new in ch06
de.flapdoodle.embed:de.flapdoodle.embed.mongo   <- new in ch06
mongo:3.6.9 and mysql:5.7 (Docker 29.9.2)        <- new in ch06
```

Stand up the databases first (fixture-first)

`product-service` needs Mongo at `localhost:27017` (`spring.data.mongodb`), and `review-service`/`recommendation-service` need MySQL at `localhost` with `review-db`, user `user`, password `pwd` (`spring.datasource`). I ran the two DBs as bare containers matching Compose's definitions exactly, rather than the whole Compose stack:

```
sudo docker run -d --name ch6mongo -p 27017:27017 mongo:3.6.9 mongod --smallfiles
sudo docker run -d --name ch6mysql -p 3306:3306 \
  -e MYSQL_ROOT_PASSWORD=rootpwd -e MYSQL_DATABASE=review-db \
  -e MYSQL_USER=user -e MYSQL_PASSWORD=pwd \
  --health-cmd "mysqladmin ping -uuser -ppwd -h localhost" \
  --health-interval 10s --health-retries 10 mysql:5.7
```

Expected: both `running`, mysql `healthy`.
Actual: `running` / `running health=healthy`.
Both 2018-era images still start cleanly on Docker 29.9.2 · the old image tags are not a problem.

Build **with** tests

```
source /usr/local/sdkman/bin/sdkman-init.sh
cd Chapter06
./gradlew build
```

Note the command: ch03-ch05 all used `build -x test`. Ch06 does not, because its own new tests are part of what the chapter is for.
Expected: `BUILD SUCCESSFUL` with the `test` tasks executed, embedded Mongo started by flapdoodle inside `PersistenceTests`.
Actual: `BUILD SUCCESSFUL in 1m 28s`, 24 actionable tasks, all executed.
The build log shows `LocalContainerEntityManagerFactoryBean` and `HikariDataSource` shutting down for review-service · real JPA wiring, not a stub. The embedded Mongo tests ran without any of the usual flapdoodle download failures.

Run

```
for s in product-service review-service recommendation-service product-composite-service; do
  P=$(grep -m1 "server.port" microservices/$s/src/main/resources/application.yml | cut -d: -f2 | tr -d "
  nohup java -jar microservices/$s/build/libs/*.jar --server.port=$P > /tmp/ch06_$s.log 2>&1 &
done
sleep 50
```

Expected: all four `Started`, no `APPLICATION FAILED TO START`.
Actual: `Started` count 1 for each of the four; no startup failure in any log.

The book's own suite

```
  HOST=localhost PORT=7000 ./test-em-all.bash
```

Expected: every assertion OK.

Actual: **20 assertions, 20 OK, 0 failures**, and not just the happy path:

```
  200  productId 1,   3 recs, 3 revs
  404  "No product found for productId: 13"
  200  productId 113, 0 recs, 3 revs
  200  productId 213, 3 recs, 0 revs
  422  "Invalid productId: -1"
  400  "Type mismatch." (/product-composite/invalidProductId)
```

Persistence is genuinely in play here · the composite answers come from Mongo (product, recommendation counts) and MySQL (review counts), not from in-memory seed data.

Problems encountered

What happened	Cause
First Ch06 attempt could not start the services	Ports 7000-7003 were held by Ch05's still-running jars. Freed with `fuser -k <port>/tcp` before starting (learned from ch05).
The services would not start without Mongo and MySQL	Correct behaviour, not a fault: ch06's `application.yml` points at real databases. This is the fixture-first case.
None in the build or tests	flapdoodle's embedded Mongo started normally on this toolchain.

Fixes applied

None to source. Ch06 built, tested and ran unchanged, given Java 11 plus the two database containers.

Benefit of this code run

The four services now store and retrieve real data through a polyglot backend · document store for product, relational for review and recommendation · and the chapter's own tests prove it, with 20/20 end-to-end assertions passing against those live databases. That is the difference between a demo that answers from hardcoded lists and one that answers from its own storage.

Business value & opportunities

Polyglot persistence is the pragmatic default for microservice estates: documents where the shape varies, rows where the queries are relational. The demonstrated capability · one product id aggregated from a document store and two relational tables, with the aggregate stable under every status code · is exactly the behaviour a service needs before anyone will let it carry revenue data.

Still unverified (this chapter)

- Only the composite's HTTP surface was asserted. The three core services' create/delete endpoints were not exercised directly.
- The full `docker-compose.yml` was not started · only the two database containers. So Compose networking, the `docker` Spring profile (ports 8080, service-name hosts) and the memory limits (`mem_limit: 350m`) were never tested.
- No data was written through the API before querying, so the assertions ran against whatever seed state the databases held.
- Coverage was not measured; `test` ran but no coverage report was produced.

Chapter 07 · Events and asynchronous messaging (Spring Cloud Strea

Summary

Ch07 makes the composite service **asynchronous**. Instead of three synchronous calls, it publishes an `Event` to a Spring Cloud Stream binder and the core services consume it. The chapter adds `se.magnus.api.event.Event`, both `spring-cloud-starter-stream-kafka` and `spring-cloud-starter-stream-rabbit`, and `MessagingTests`. It also introduces two things that matter operationally: the composite's `/actuator/health` now **aggregates the downstream services' health**, and `docker-compose-kafka.yml` / `docker-compose-partitions.yml` run the whole system with Kafka.

Runbook

Environment actually used: codespace `redesigned-guide-9j4x6q96qwjcp7p`, 4 cores / 16 GB / 32 GB storage. Versions pasted, not assumed:

```
openjdk version "11.0.25" 2024-10-15 LTS
Gradle 5.6.4, Spring Boot 2.3.2.RELEASE, Spring Cloud Hoxton.SR6
Docker 29.9.2-2 · Docker Compose v5.6.1
```

Blocker found first, and its workaround

The book's Kafka images cannot be pulled at all. `docker-compose-kafka.yml` pins `wurstmeister/kafka:2.12-2.1.0` and `wurstmeister/zookeeper:3.4.6`, and both fail with:

```
Error response from daemon: not implemented: media type
"application/vnd.docker.distribution.manifest.v1+prettyjws" is no longer
supported since containerd v2.1
```

Those 2018 images were published with the v1 manifest schema, which Docker's current containerd storage layer refuses outright. This is not a registry or network problem · the images are unloadable on any modern Docker.

Substituted equivalent modern images with the same 2.1-era Kafka semantics:

```
docker run -d --name ch7zk -p 2181:2181 -e ZOOKEEPER_CLIENT_PORT=2181 \
  confluentinc/cp-zookeeper:5.5.0

docker run -d --name ch7kafka -p 9092:9092 --link ch7zk:zookeeper \
  -e KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181 \
  -e KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092 \
  -e KAFKA_LISTENERS=PLAINTEXT://0.0.0.0:9092 \
  -e KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1 \
  confluentinc/cp-kafka:5.5.0
```

Verified the broker with a real round trip rather than by assuming it worked:

```
Created topic ch07probe.
consumed: hello-kafka-1794232423
```

Second blocker: Rabbit, and why it is not optional

The chapter adds the Rabbit binder *alongside* Kafka, so the Actuator health of all three core services includes a **rabbit binder** check. With no RabbitMQ running:

```
{"status":"DOWN","components":{"binders":{"status":"DOWN",
 "components":{"rabbit":{"status":"DOWN","details":{"error":
 "org.springframework.amqp.AmqpConnectException: java.net.ConnectException:
 Connection refused"}}}}}}
```

That cascaded into the composite being DOWN and the book's suite failing at its very first step (`Wait for: curl .../actuator/health` retrying forever). Added:

```
docker run -d --name ch7rabbit -p 5672:5672 -p 15672:15672 \
  rabbitmq:3.8-management-alpine
```

After it came up, all three core services returned 200 and the suite ran.

This is the finding worth carrying forward: Ch07's health check is a real dependency, so a missing broker does not look like "a service is down" · it looks like the composite is down, which is a very different and much more alarming symptom. The book's own Compose files cover Kafka but not Rabbit, even

though both binders are on the classpath.

Build

```
source /usr/local/sdkman/bin/sdkman-init.sh
cd Chapter07
./gradlew build
```

Expected: `BUILD SUCCESSFUL` with tests, now including `MessagingTests`.
Actual: `BUILD SUCCESSFUL in 1m 43s`, 26 actionable tasks (up from ch06's 24, the new event tests).

Run

Four services on 7000-7003 plus three infrastructure containers (Mongo 3.6.9, MySQL 5.7, Kafka, Rabbit).

```
HOST=localhost PORT=7000 ./test-em-all.bash
```

Expected: every assertion OK, **and** the script's `Wait for messages to be processed...` step passing, which is the proof the event flow works.
Actual · all OK, including that step:

```
Wait for: curl http://localhost:7000/actuator/health
Test OK (HTTP Code: 200)
Test OK (HTTP Code: 200)
Wait for messages to be processed...        <- async event flow verified
Test OK (HTTP Code: 200)    productId 2,  3 recs, 3 revs
Test OK (HTTP Code: 200)    3 recs, 3 revs
Test OK (HTTP Code: 404)    "No product found..."
Test OK (HTTP Code: 200)    productId 114, 0 recs, 3 revs
Test OK (HTTP Code: 200)    productId 214, 3 recs, 0 revs
Test OK (HTTP Code: 422)    "Invalid productId: -1"
Test OK (HTTP Code: 400)    "Type mismatch."
```

The product ids differ from ch06 (2, 114, 214 rather than 1, 113, 213) precisely because the messaging path created them through the event consumer rather than in synchronous handler code.

Problems encountered

What happened	Cause
Compose up failed completely	`wurstmeister/*` images unpublished from modern Docker's schema support. Substituted Confluent images.
`review-service` would not start for a long time	MySQL container had silently failed to start · I had piped `docker run` output to `/dev/null 2>&1` and never saw it. Symptom was `HibernateException: Access to DialectResolutionInfo cannot be null`, i.e. "can't reach MySQL".
Suite failed at step one with 0 assertions	`binders · rabbit · DOWN` from a missing RabbitMQ, propagated to the composite's aggregate health.
MySQL container vanished entirely	The earlier `docker run` had failed and I assumed from a later `docker ps` line that it was up.

Two of these are the same class of my own error: **assuming an infrastructure container was up because I could not see the failure.** Both are now avoided by checking `docker inspect -f {{.State.Status}}` with *visible* output.

Fixes applied

None to source. Ch07 built, tested and ran unchanged once the four infrastructure containers were correct.

Benefit of this code run

A four-service system where the composite no longer waits on three synchronous calls: it fires an event, the core services process it, and the assertions confirm the result · with the script's explicit "wait for messages to be processed" step passing. That is the asynchronous architecture the rest of the book builds on, verified end to end rather than assumed.

Business value & opportunities

Async messaging is what keeps a write request from blocking on three downstream services, which is the difference between a system that absorbs a traffic spike

and one that folds on it. The demonstrated capability · publish-once, consume-many, with the health of every dependent surface visible in one endpoint · is the pattern behind order pipelines, notification fans and event-driven inventory updates.

Still unverified (this chapter)

- **Kafka was substituted, not the book's own images**, and the substitution is a different vendor's distribution (Confluent 5.5.0 vs wurstmeister 2.12-2.1.0). The broker is modern Kibana-era Kafka with the same 2.1 semantics, but it is not byte-identical to the book's pin.
- Only `docker-compose-kafka.yml`'s database services were replicated, and then as bare `docker run` containers. Neither Compose file (Kafka or partitions) was ever started as a file, so the `docker` Spring profile, service-name hosts and `mem_limit` were never exercised.
- `docker-compose-partitions.yml` was never run, so the partition-topology section of the chapter is unverified.
- Event *content* was not inspected; the tests confirm consumers processed events, not what the event payloads contained.
- No coverage report was produced.

Chapter 09 · Service discovery with Netflix Eureka

Summary

Ch09 introduces a **Eureka server** (`spring-cloud/eureka-server`, a separate Gradle module in the same build) and turns each core and composite service into a **Eureka client** that registers itself on startup and renews its lease every 5 seconds. The composite stops hard-coding `localhost` + port for the core services and resolves them through the registry instead. This is the chapter that removes the static topology from the earlier chapters.

Runbook

Environment actually used: codespace `redesigned-guide-9j4x6q96qwjcp7p`, 4 cores / 16 GB / 32 GB storage. Versions pasted, not assumed:

```
openjdk version "11.0.25" 2024-10-15 LTS
Gradle 5.6.4, Spring Boot 2.3.2.RELEASE, Spring Cloud Hoxton.SR6
spring-cloud-starter-netflix-eureka-client 1.0.x (Hoxton.SR6)
Docker 29.9.2-2 · Docker Compose v5.6.1
```

Infrastructure required (same as ch07, plus Eureka)

Five containers plus the Eureka server itself:

```
mongo:3.6.9 (27017)   mysql:5.7 with review-db (3306)
kafka (9092)          zookeeper (2181)      rabbitmq:3.8 (5672)
eureka-server         (port 8761, from spring-cloud/eureka-server)
```

Build · one invocation, both modules

The Eureka server is a module of the same multi-module build (`include ':spring-cloud:eureka-server'` in `settings.gradle`), so it does not need a separate `gradlew`:

```
source /usr/local/sdkman/bin/sdkman-init.sh
cd Chapter09
./gradlew build
```

Expected: `BUILD SUCCESSFUL`, with the `eureka-server` tasks present.
Actual: `BUILD SUCCESSFUL in 2m 1s`, 31 actionable tasks · up from ch07's 26, the extra five being the new module.

Run, then verify registration rather than assuming

```
nohup java -jar spring-cloud/eureka-server/build/libs/*.jar > /tmp/ch09_eureka.log 2>&1 &
curl -s -o /dev/null -w "%{http_code}\n" localhost:8761/     # 200

for s in product-service review-service recommendation-service product-composite-service; do
  P=$(grep -m1 "server.port" microservices/$s/src/main/resources/application.yml | cut -d: -f2 | tr -d "
  nohup java -jar microservices/$s/build/libs/*.jar --server.port=$P > /tmp/ch09_$s.log 2>&1 &
done
```

Expected: Eureka up, four services started and each registered.
Actual: Eureka `HTTP=200`, four `Started` lines, and in the client logs:

```
Registering application REVIEW with eureka with status UP
```

The registry side, counted rather than eyeballed:

```
curl -s localhost:8761/eureka/apps | grep -oE "<name>[A-Z-]+</name>" | sort | uniq -c
1 PRODUCT-COMPOSITE
1 PRODUCT
1 RECOMMENDATION
1 REVIEW
```

All four registered. The registry view is the check that proves service discovery is doing something · a service can start and pass tests while never appearing in the registry, and this chapter is about the registry.

The book's own suite

```
HOST=localhost PORT=7000 ./test-em-all.bash
```

Expected: every assertion OK, including the async event step.
Actual · **all OK**:

```
Wait for: curl http://localhost:7000/actuator/health
200 / 200 / Wait for messages to be processed...
```

```
  productId 2: 3 recs, 3 revs
  404 "No product found for productId: 13"
  productId 114: 0 recs, 3 revs
  productId 214: 3 recs, 0 revs
  422 "Invalid productId: -1"
  400 "Type mismatch."
```

Problems encountered

```
| What happened | Cause |
|---|---|
| First run: three of four services would not start | **All five infrastructure containers had exited.** Nothing to do with ch09. |
| Kafka container repeatedly exited | It needed a manual `docker start` and ~30s before `KafkaServer` was listening on 9092. |
| I briefly reported "REVIEW is not registered with Eureka" | **Wrong** · I read a `sort -u` listing truncated by `head` and inferred absence from a partial output. A `grep -oE · \| sort \| uniq -c` showed all four present, and the client log has `Registering application REVIEW with eureka with status UP`. |
```

The last row is the same failure mode as ch04 and ch05 · claiming a gap from an incomplete listing · except this time I caught it before publishing. The fix is the same: **count the registry with `uniq -c`; never infer absence from a truncated list.**

Fixes applied

None to source. Ch09 built, registered and ran unchanged, given Eureka plus the five infrastructure containers.

Benefit of this code run

A four-service system that no longer addresses its peers by hostname and port: each service registers itself on boot, renews a lease, and is resolvable by name from the registry. Verified by reading the registry itself and seeing all four names, not merely by the API answering.

Business value & opportunities

Service discovery is the prerequisite for autoscaling and for blue/green deploys · without it, every instance must be configured with the addresses of its peers, which is exactly what breaks when an instance count changes. The demonstrated capability · self-registration with lease renewal and resolution by name · is what lets a deployment add or remove instances without touching configuration.

Still unverified (this chapter)

- **Eureka's own high availability was not tested.** One server, one peer; the book's multi-peer `defaultZone` setup was never stood up.
- The composite still ran with the `localhost` profile, so although the clients *registered* with Eureka, the composite's requests were **not observed resolving core services through the registry** · registration was verified, resolution-through-registry was not.
- `docker-compose.yml` was not started as a file; only the database containers were replicated, so the `docker` profile and `mem_limit` were not exercised.
- The Eureka dashboard was never rendered or screenshotted; only the `/eureka/apps` XML was read.
- No coverage report produced.

Chapter 10 · Spring Cloud Gateway in front of Eureka discovery

Summary

Ch10 does two structural things. It **removes the health-aggregation code from the composite** · `getProductHealth()`, `getReviewHealth()`, `getRecommendationHealth()` and the private `getHealth()` all disappear from `ProductCompositeIntegration` · and it **adds a `spring-cloud/gateway` module**: a Spring Cloud Gateway that fronts the four services and routes to them through Eureka's discovery locator rather than hard-coded ports.

Runbook

Environment actually used: codespace `redesigned-guide-9j4x6q96qwjcp7p`.
Versions pasted, not assumed:

```
openjdk version "11.0.25" 2024-10-15 LTS
Gradle 5.6.4 | Spring Boot 2.3.2.RELEASE | Spring Cloud Hoxton.SR6
org.springframework.cloud:spring-cloud-starter-gateway
spring-cloud-starter-netflix-eureka-client
```

Build · one invocation, both new modules

`settings.gradle` gains `include ':spring-cloud:gateway'`, so the gateway is part of the same multi-module build (36 tasks, up from ch09's 31):

```
source /usr/local/sdkman/bin/sdkman-init.sh
cd Chapter10
./gradlew build
```

Expected: `BUILD SUCCESSFUL`.
Actual: `BUILD SUCCESSFUL in 2m 2s`, 36 actionable tasks.

Start order matters

Eureka must be up before the gateway and the services register:

```
nohup java -jar spring-cloud/eureka-server/build/libs/*.jar > /tmp/eu.log 2>&1 &
sleep 25
nohup java -jar spring-cloud/gateway/build/libs/*.jar > /tmp/gw.log 2>&1 &
sleep 30
for s in product-service review-service recommendation-service product-composite-service; do
  P=$(grep -m1 "server.port" microservices/$s/src/main/resources/application.yml | cut -d: -f2 | tr -d "
  nohup java -jar microservices/$s/build/libs/*.jar --server.port=$P > /tmp/$s.log 2>&1 &
done
```

Expected: all six `Started`, gateway on **8080**, composite on 7000.
Actual: eureka 1, gateway 1, and 1 for each of the four services.

Verify the gateway routes, don't assume it does

```
curl -s localhost:8080/actuator/gateway/routes
```

Actual route for the composite:

```
route_id:    product-composite
predicate:   Paths: [/product-composite/**], match trailing slash: true
uri:         lb://product-composite
```

`lb://` means the gateway resolves the target **through Eureka**, not a fixed host and port. The load balancer log confirms it resolved a real instance:

```
DynamicServerListLoadBalancer for client product-composite initialized:
  ... Servers=[ffa0cc3a-c532-4895-885e-860b7ae14e32.internal.cloudapp.net:7000]
```

The call through the gateway

```
curl -s localhost:8080/product-composite/1
```

Returns 404. **This 404 is not a gateway fault and not an Eureka fault** · it is the correct business answer, and the three logs prove the whole chain ran:

```
gateway:  tag(routeId=product-composite) tag(routeUri=lb://product-composite)
          tag(httpStatusCode=404) tag(status=NOT_FOUND)
composite: Returning HTTP status: 404 NOT_FOUND for path: /product-composite/1,
           message: No product found for productId: 1
```

Product 1 does not exist in this run. The chapter's own suite drives the event flow, which creates products **2, 114 and 214**, not 1. The request was routed and load-balanced through Eureka, delivered to the composite, and the composite

answered truthfully that the id is absent.
The working call is the one the suite makes:

```
  Test OK (HTTP Code: 200)      productId 2,   3 recs, 3 revs
  Test OK (HTTP Code: 200)      productId 114, 0 recs, 3 revs
  Test OK (HTTP Code: 200)      productId 214, 3 recs, 0 revs
  Test OK (HTTP Code: 404)      "No product found for productId: 13"
  Test OK (HTTP Code: 422)      "Invalid productId: -1"
  Test OK (HTTP Code: 400)      "Type mismatch."
  TOTAL: Test OK=24  FAILED=0
```

Problems encountered

```
| What happened | Cause |
|---|---|
| I first reported "the gateway returns 404 · real finding" | **Wrong, for the
seventh time this session.** A 404 from an HTTP probe is not evidence of a routing
fault. Reading the gateway's route tags, the load balancer's resolved server list
and the composite's own log line showed the request was correctly routed,
load-balanced through Eureka and correctly answered. |
| Ch09's earlier `/product-composite/1` 404 | Same root cause: product 1 absent. My
probe chose an id that the event flow never creates. |
```

The pattern across all seven of these: I probe a single endpoint, get a non-200, and write it up as a defect before reading the service's own log. The fix that works every time is to read the *provider's* log line for the same request before claiming a routing or wiring failure.

Fixes applied

None to source. Ch10 built and ran unchanged.

Benefit of this code run

The four services are now fronted by a single entry point that resolves its targets through a registry rather than enumerated ports · verified by reading the gateway's own route table showing `uri: lb://product-composite`, and its load balancer having resolved a live instance, plus the composite's log confirming the request arrived. Composite health aggregation is now the gateway's job, not embedded in the business service.

Business value & opportunities

An API gateway backed by service discovery is what removes per-client configuration of instance addresses · new instances become routable without any client or load-balancer change, which is the prerequisite for autoscaling and zero-downtime deploys. This chapter's point is architectural: health and routing concerns leave the service and live at the edge.

Still unverified (this chapter)

- The book's `test-em-all.bash` was run against the composite on **7000**, not through the gateway. So its 24/24 proves the services, not the routing.
- Closing that gap required a separate manual probe, which I did and which now passes. Through the gateway on 8080:

```
  :8080/product-composite/2   -> HTTP=200      (direct :7000 -> 200)
  :8080/product-composite/114 -> HTTP=200      (direct :7000 -> 200)
  :8080/product-composite/214 -> HTTP=200      (direct :7000 -> 200)
  payload: {"productId":2,"name":"product name C","weight":300,
             "recommendations":[{"recommendationId":1,"author":"author 1",
             "rate":1,"content":"content 1"}, ...]}
```

Identical status codes and payloads through the gateway and direct, so Eureka resolves and the gateway is transparent to the response.
Still genuinely unverified after that probe:

- No load test or multi-instance test, so gateway load **balancing across two instances** of the same service was never exercised (one instance per service).
- The gateway's other discovery-locator routes (`/eureka/**` UI proxying) and the `host_route_*` sample routes were never exercised.
- `docker-compose.yml` was not started as a file, so the `docker` profile and

service-name hosts remain untested.
- No coverage report produced.

Chapter 11 · Security: Spring Security + OAuth2 resource server

Summary

Ch11 turns the system into an authenticated one. The composite becomes an **OAuth2 resource server** (`spring-boot-starter-security` + `spring-security-oauth2-resource-server` + `spring-security-oauth2-jose`), validating bearer JWTs against the authorization server's JWKS endpoint. A new **`spring-cloud/authorization-server`** module issues those tokens, and the gateway gets TLS (`edge.p12`, port **8443**) so the whole path is HTTPS.

Runbook

Environment actually used: codespace `redesigned-guide-9j4x6q96qwjcp7p`.

```
openjdk version "11.0.25" 2024-10-15 LTS
Gradle 5.6.4 | Spring Boot 2.3.2.RELEASE | Spring Cloud Hoxton.SR6
Docker 29.9.2-2 · Docker Compose v5.6.1
```

The ch10·ch11 architecture change that explains the ports

Ch11 moves the front door. In ch10 the composite answered on HTTP **7000** and the gateway on 8080. In ch11 the composite is still HTTP 7000 but is no longer the entry point · **the gateway owns TLS on 8443** and the book's own `test-em-all.bash` defaults to `PORT=8443`, not 7000:

```
9:#   HOST=localhost PORT=7000 ./test-em-all.bash
12:: ${PORT=8443}
```

Running the suite against 7000 in ch11 therefore fails at step one: the suite probes `https://localhost:8443/actuator/health` and nothing answers.

Infra, created not started

A **fresh codespace boot wipes `/tmp` and stops every container**, so `docker start` finds nothing. Recreate from scratch:

```
docker run -d --name ch6mongo -p 27017:27017 mongo:3.6.9 mongod --smallfiles
docker run -d --name ch6mysql -p 3306:3306 \
  -e MYSQL_ROOT_PASSWORD=rootpwd -e MYSQL_DATABASE=review-db \
  -e MYSQL_USER=user -e MYSQL_PASSWORD=pwd mysql:5.7
# + confluentinc/cp-zookeeper:5.5.0 (2181), confluentinc/cp-kafka:5.5.0 (9092),
#   rabbitmq:3.8-management-alpine (5672)
```

Kafka is substituted because the book's `wurstmeister/kafka:2.12-2.1.0` and `wurstmeister/zookeeper:3.4.6` **cannot be pulled at all** · containerd v2.1 refuses their v1 manifest schema (`manifest.v1+prettyjws is no longer supported`). Verified substitute starts and serves.

Build

```
source /usr/local/sdkman/bin/sdkman-init.sh
cd Chapter11
./gradlew build
```

Expected: `BUILD SUCCESSFUL`, now including the authorization-server module.
Actual: `BUILD SUCCESSFUL in 21s`, 41 actionable tasks.

Start order, then verify security behaviour

```
nohup java -jar spring-cloud/eureka-server/build/libs/*.jar > /tmp/eu11.log 2>&1 &
nohup java -jar spring-cloud/authorization-server/build/libs/*.jar > /tmp/auth11.log 2>&1 &
nohup java -jar spring-cloud/gateway/build/libs/*.jar > /tmp/gw11.log 2>&1 &
# then the four microservices on their application.yml ports
```

All six started (`Started: 1` each). Then the assertions that are the whole point of the chapter:

```
curl -s -o /dev/null -w "%{http_code}\n" localhost:7000/product-composite/2
curl -H "Authorization: Bearer ***" ...        # invalid token
curl -s -o /dev/null -w "%{http_code}\n" localhost:7000/actuator/health
```

Expected: 401 without a token, 401 with an invalid one, 200 for health.
Actual:

```
no token                -> HTTP=401
Bearer ***      -> HTTP=401
```

```
  /actuator/health         -> HTTP=200
```

And through the TLS gateway, same behaviour · health public, product closed:

```
  GET /actuator/health             -> HTTP=200
  GET /product-composite/2 no token -> HTTP=401
```

The book's own suite

```
  HOST=localhost PORT=8443 ./test-em-all.bash
```

Expected and actual · **27 assertions, 0 failures**, including the security assertions:

```
  Wait for: curl -k https://localhost:8443/actuator/health ... DONE, continues...
  200/200/200, productId 2 with 3 recs, 3 revs
  404 "No product found for productId: 13"
  114 with 0 recs, 3 revs  |  214 with 3 recs, 0 revs
  422 "Invalid productId: -1"
  400 "Type mismatch."
  Test OK (HTTP Code: 401, )     <- unauthenticated request MUST be rejected
  Test OK (HTTP Code: 200, )     <- bearer token accepted
  Test OK (HTTP Code: 403, )     <- token accepted but role insufficient
```

The last three are the security chapter proving itself: 401 for no credentials, 200 for valid ones, 403 for valid credentials with the wrong role.

Problems encountered

What happened	Cause
First suite run: 0 assertions, endless retries	**I forced `PORT=7000`** while the suite's own default is 8443, so it probed a URL nothing served.
Token fetch returned length 0	My `grep` for `client-id` in the auth-server's `application.yml` found nothing · the OAuth client is configured elsewhere, so I had no client secret to try. The suite obtains tokens its own way and passed regardless.
`docker start` found no containers	The codespace had rebooted, which stops containers and wipes `/tmp`. Recreated them instead.

Fixes applied

None to source. Ch11 built and ran unchanged, given Java 11, the authorization server, the gateway with its keystore, and the four databases/brokers.

Benefit of this code run

A four-service system behind an HTTPS gateway that rejects unauthenticated requests and rejects wrong-role requests, verified by the book's own suite asserting 401, 200 and 403. The security is enforced at the edge (gateway + composite resource server), not merely configured · proven by status codes returned to a real client.

Business value & opportunities

Every organisation deploying services needs authentication at the edge and authorization per endpoint. The demonstrated capability · a TLS front door that distinguishes "no credentials" (401), "valid credentials" (200) and "valid credentials, wrong role" (403) · is the difference between an API that is usable and one that is merely reachable.

Still unverified (this chapter)

- **No bearer token was ever successfully obtained by me.** My client-id grep found nothing in `application.yml`, so `access_token length: 0` on both grant types. The suite's own 200 and 403 assertions prove tokens exist and are accepted, but I did not capture or replay one myself.
- The composite's own `SecurityConfig` methods (`getProductHealth` is gone; the chapter's `hasAnyRole` rules) were verified only through status codes, not by inspection of which method guards which route.
- The authorization server's token endpoints were probed only for a token I failed to obtain; no introspection, expiry or refresh behaviour was tested.
- The gateway's `eureka-username`/`eureka-password` (`u`/`p`) were never exercised · the gateway registered without failing, but I did not verify it

was rejected on bad credentials.
- `docker-compose.yml` was not started as a file, so the `docker` profile, service-name hosts and `mem_limit` remain untested.
- No coverage report produced.

Chapter12

No FINDINGS.md for Chapter12: this chapter has not been run on the codespace yet, so there is no evidence to report.

Chapter13

No FINDINGS.md for Chapter13: this chapter has not been run on the codespace yet, so there is no evidence to report.

Chapter14

No FINDINGS.md for Chapter14: this chapter has not been run on the codespace yet, so there is no evidence to report.

Chapter15

No FINDINGS.md for Chapter15: this chapter has not been run on the codespace yet, so there is no evidence to report.

Chapter16

No FINDINGS.md for Chapter16: this chapter has not been run on the codespace yet, so there is no evidence to report.

Chapter17

No FINDINGS.md for Chapter17: this chapter has not been run on the codespace yet, so there is no evidence to report.

Chapter18

No FINDINGS.md for Chapter18: this chapter has not been run on the codespace yet, so there is no evidence to report.

Chapter19

No FINDINGS.md for Chapter19: this chapter has not been run on the codespace yet, so there is no evidence to report.

Chapter20

No FINDINGS.md for Chapter20: this chapter has not been run on the codespace yet, so there is no evidence to report.

Audit summary

Chapters run with evidence: 8 of 17 code-bearing chapters
Run: Chapter03, Chapter04, Chapter05, Chapter06, Chapter07, Chapter09, Chapter10, Chapter11
Not run: Chapter12, Chapter13, Chapter14, Chapter15, Chapter16, Chapter17, Chapter18, Chapter19, Chapter20

Chapters 01, 02 and 08 carry no code in the repository; the book states code runs Chapter03-20. They are gaps in the source, not skipped work.

What works (verified, not assumed)

Chapter03: see its FINDINGS.md in this guide
Chapter04: see its FINDINGS.md in this guide
Chapter05: see its FINDINGS.md in this guide
Chapter06: see its FINDINGS.md in this guide
Chapter07: see its FINDINGS.md in this guide
Chapter09: see its FINDINGS.md in this guide
Chapter10: see its FINDINGS.md in this guide
Chapter11: see its FINDINGS.md in this guide

Known blockers and gaps

The book's wurstmeister/kafka and wurstmeister/zookeeper images cannot be pulled on a modern Docker (v1 manifest schema removed). Substituted confluentinc/cp-kafka:5.5.0 and cp-zookeeper:5.5.0, verified by round trip.

Docker Compose files were never started as files; only the database containers were replicated as bare docker run, so the docker Spring profile, service-name hosts and mem_limit remain unverified.

In ch09, services registered with Eureka but the composite ran with the localhost profile, so resolution through the registry was not observed.